ASD&D LABORATORY REPORT

Experiment 4: Jenkins Installation and Jenkins Pipeline

Subject: Agile Software Development & DevOps (ASD&D)

Department: Artificial Intelligence & Data Science

2. Experiment 4.1 — Continuous Integration and Jenkins Installation

2.1 Aim

The objective of this experiment was to develop a conceptual understanding of Continuous Integration (CI) as a software engineering practice and to perform the installation and initial configuration of Jenkins, an open-source automation server widely used for implementing CI/CD workflows.

2.2 Learning Outcome

LO3: Ability to apply DevOps tools and practices to automate the software build, integration, and deployment lifecycle.

2.3 Introduction to Continuous Integration

Continuous Integration is a software development practice in which developers frequently merge their code changes into a shared central repository, after which automated builds and tests are triggered. This practice promotes early detection of integration errors, encourages close collaboration among team members, and ensures that the codebase remains in a consistently deployable state. By automating the build and test cycle, Continuous Integration significantly reduces the time and effort required to identify and resolve defects, thereby enhancing both the quality and efficiency of the overall software development process.

2.4 Introduction to Jenkins

Jenkins is a widely adopted open-source automation server that facilitates Continuous Integration and Continuous Delivery (CI/CD) of software projects. It enables development teams to automate various stages of the software development lifecycle, including source code compilation, automated testing, packaging, deployment, and monitoring. Jenkins supports an extensive ecosystem of plugins and third-party integrations, which allows it to be configured for a broad range of automation tasks across diverse technology stacks. Owing to its flexibility, extensibility, and active community support, Jenkins has become one of the most widely used automation servers in modern DevOps environments.

2.5 Installation Procedure

The Jenkins installation was carried out in a stepwise manner, as detailed below.

1. Java Installation: The Java Development Kit (JDK) was installed on the system, as Jenkins is a Java-based application and requires a compatible Java Runtime Environment to execute.

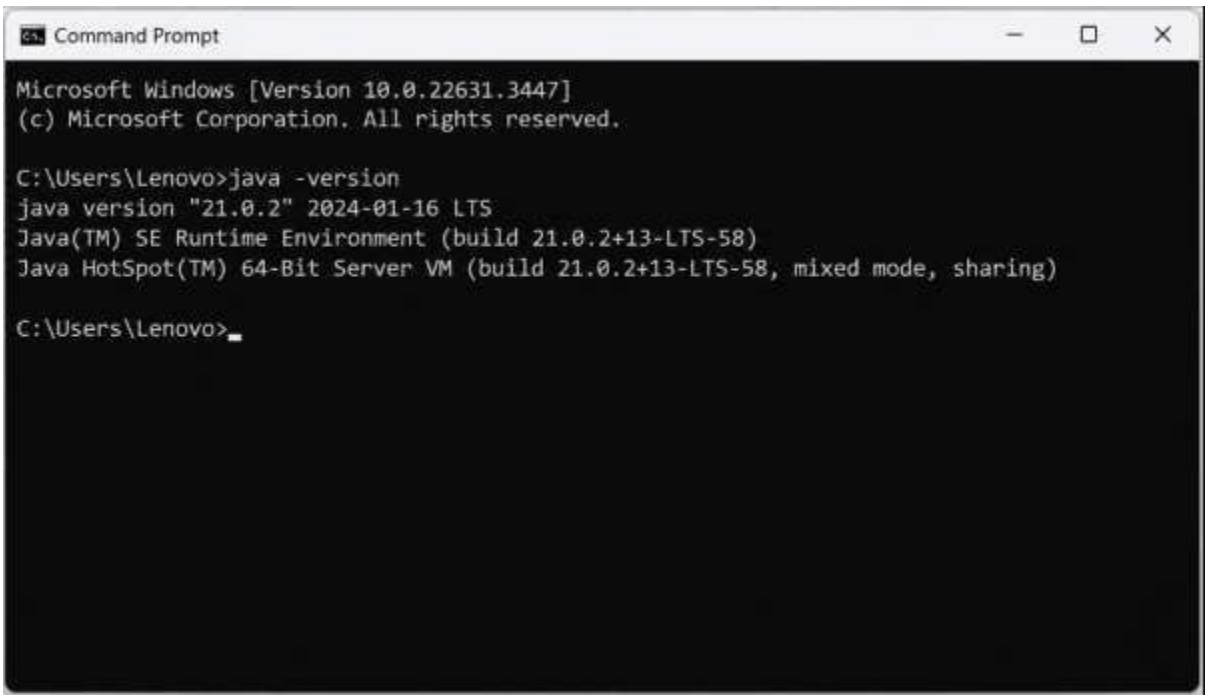


Figure 2.1: Verification of Java Development Kit installation

2. Download Jenkins: The latest Jenkins WAR (Web Application Archive) file was downloaded from the official Jenkins website.

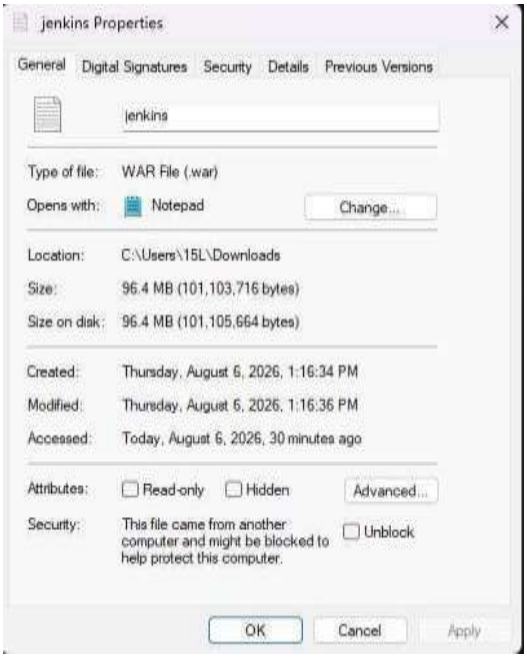


Figure 2.2: Jenkins WAR file download from the official website

3. Launch Terminal: A terminal (command prompt) session was opened on the local system to execute the Jenkins startup command.

```
java -jar C:\Users\<User>\Downloads\jenkins.war --enable-future-java
```

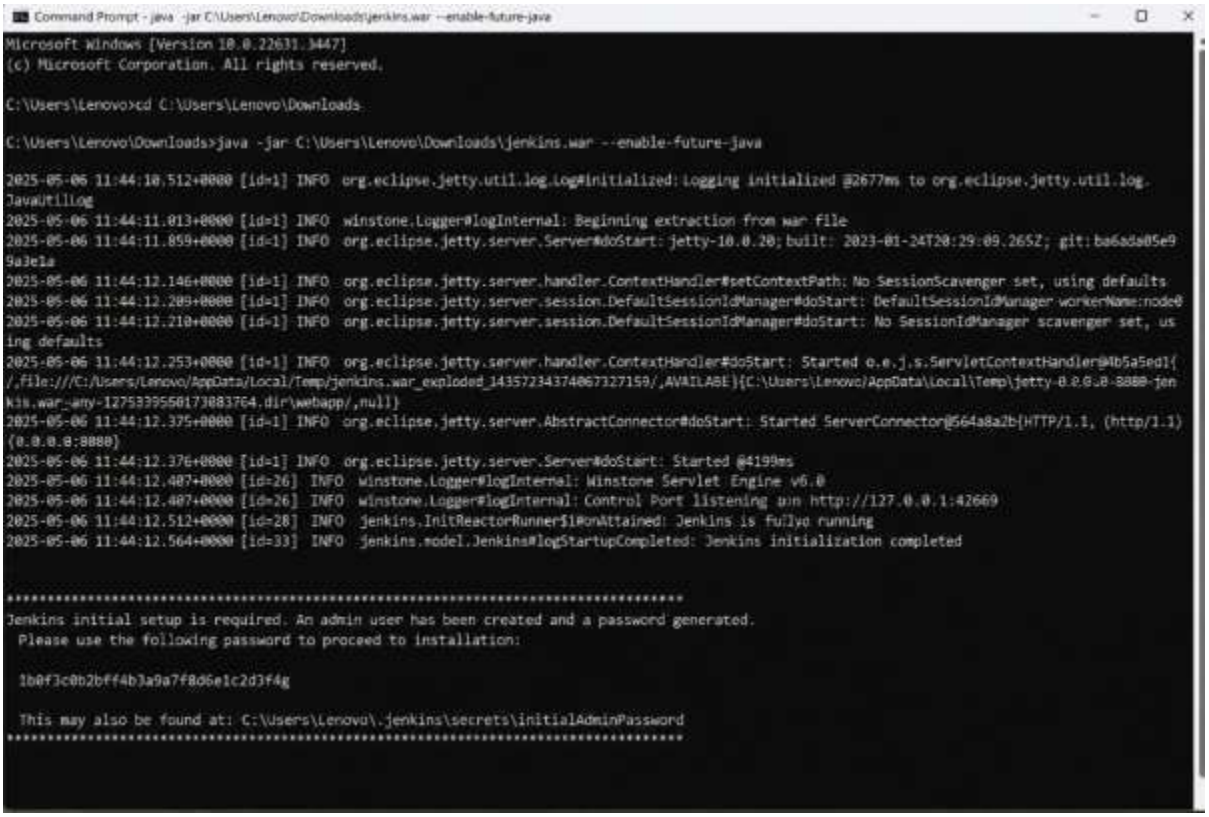


Figure 2.3: Jenkins server startup command executed in the terminal

4. Navigate to Directory: The cd command was used to navigate to the directory containing the downloaded Jenkins WAR file, ensuring the server could be launched from the correct working directory.



Figure 2.4 & 2.5: Navigation to the Jenkins WAR file directory

5. Start Jenkins: The command java -jar jenkins.war was executed to initiate the Jenkins server process.
6. Access Web UI: A web browser was opened and directed to http://localhost:8080 to access the Jenkins setup wizard.

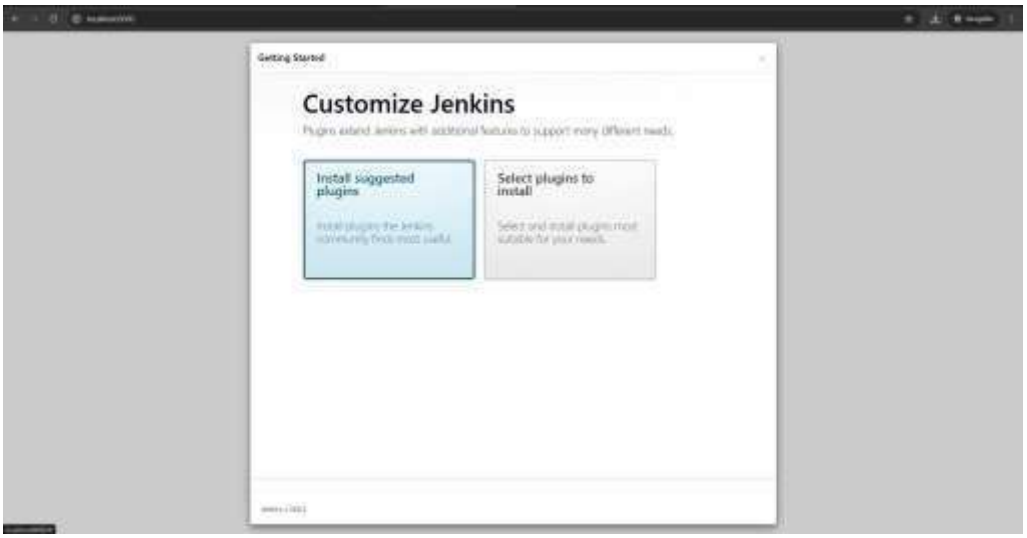


Figure 2.6: Jenkins setup wizard accessed via the web browser

7. Unlock Jenkins: The initial administrator password, generated in the terminal during server startup, was retrieved and entered into the setup wizard to unlock Jenkins.
8. Customize Plugins: The recommended set of plugins was selected for installation, ensuring that the commonly required Jenkins functionalities were made available.
9. Create Admin User: An administrator account was created by specifying the required credentials, enabling secure access to the Jenkins environment.
10. Jenkins URL: The Jenkins instance URL was confirmed, and the initial setup process was completed.
11. Dashboard: The Jenkins dashboard was accessed successfully by logging in with the configured administrator credentials.

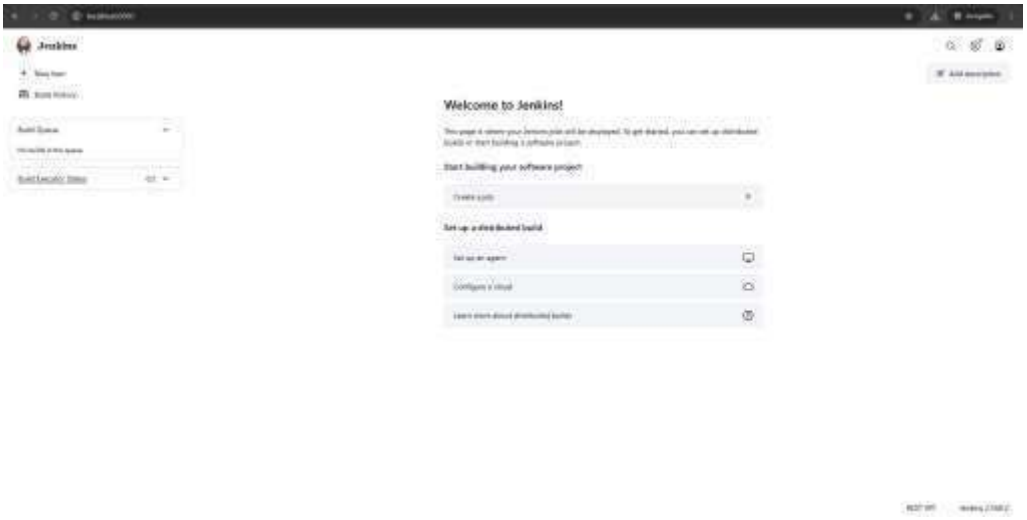


Figure 2.7: Jenkins dashboard displayed after successful login

2.6 Observations

During the installation process, it was observed that Jenkins requires a properly configured Java environment as a prerequisite for successful execution. The setup wizard provided a

guided, user-friendly workflow for plugin selection, administrator account creation, and initial server configuration. Upon completion of the setup, the Jenkins dashboard was found to be fully accessible and operational, confirming that the automation server had been correctly installed and configured.

2.7 Result

The Jenkins automation server was successfully installed and configured on the local system. The Jenkins dashboard was accessed without errors, verifying that the installation process was completed correctly and that the server was ready for pipeline configuration and job execution.

2.8 Conclusion

Through this experiment, a clear understanding of Continuous Integration as a software development practice was developed, along with practical, hands-on experience in installing and configuring the Jenkins automation server. The successful completion of the installation process demonstrated the feasibility of using Jenkins as a foundation for implementing automated CI/CD pipelines in subsequent experiments.

3. Experiment 4.2 — Jenkins Pipeline Implementation

3.1 Aim

The objective of this experiment was to design, implement, and execute a Jenkins Pipeline for automating the build, test, and deployment stages of a software project.

3.2 Learning Outcome

LO1, LO3: Ability to design automated CI/CD workflows and apply DevOps tools to implement reliable, repeatable software delivery pipelines.

3.3 Introduction to Jenkins Pipeline

A Jenkins Pipeline is a suite of automated processes that enables developers and DevOps teams to build, test, and deploy code within a Continuous Integration/Continuous Delivery (CI/CD) environment. Fundamentally, a pipeline represents a workflow that defines how software progresses from source code to production in an automated and repeatable manner. Pipelines are defined using a domain-specific language based on Groovy and are typically stored as code within a file named Jenkinsfile, which can be version-controlled alongside the project source code.

3.4 Benefits of Pipeline

- Automates the end-to-end CI/CD process, minimizing manual intervention.
- Provides improved visibility into each stage of the build and deployment process.
- Enables version control of build logic through the Pipeline-as-Code approach.
- Supports complex workflows, including parallel execution and conditional logic.

3.5 Types of Pipelines

Jenkins supports two principal syntax styles for defining pipelines, namely the Declarative Pipeline and the Scripted Pipeline.

3.5.1 Declarative Pipeline

The Declarative Pipeline offers a simpler and more structured syntax, designed to provide an easy-to-read format for defining CI/CD workflows. It was introduced to improve the readability and maintainability of pipeline definitions.

Key characteristics of the Declarative Pipeline include:

- Utilizes the pipeline {} block as the top-level structure.
- Organized into well-defined stages and steps.
- Provides simplified and more robust error handling.
- Offers enhanced syntax validation within the Jenkins user interface.

3.5.2 Scripted Pipeline

The Scripted Pipeline offers greater flexibility and control, as it is written using Groovy-based scripting syntax. It is generally suited to advanced users who require fine-grained control over the build process.

Key characteristics of the Scripted Pipeline include:

- Utilizes the node {} block as the top-level structure.
- Defines the entire workflow programmatically as code.
- Permits complex scripting logic and conditional branching.
- Presents a comparatively steeper learning curve for beginners.

3.6 Pipeline Implementation

3.6.1 Declarative Pipeline — Creation

The following Declarative Pipeline script was created and executed within the Jenkins environment:

```
pipeline {
    agent any

    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins!'
            }
        }
    }
}
```

3.6.2 Output / Observation

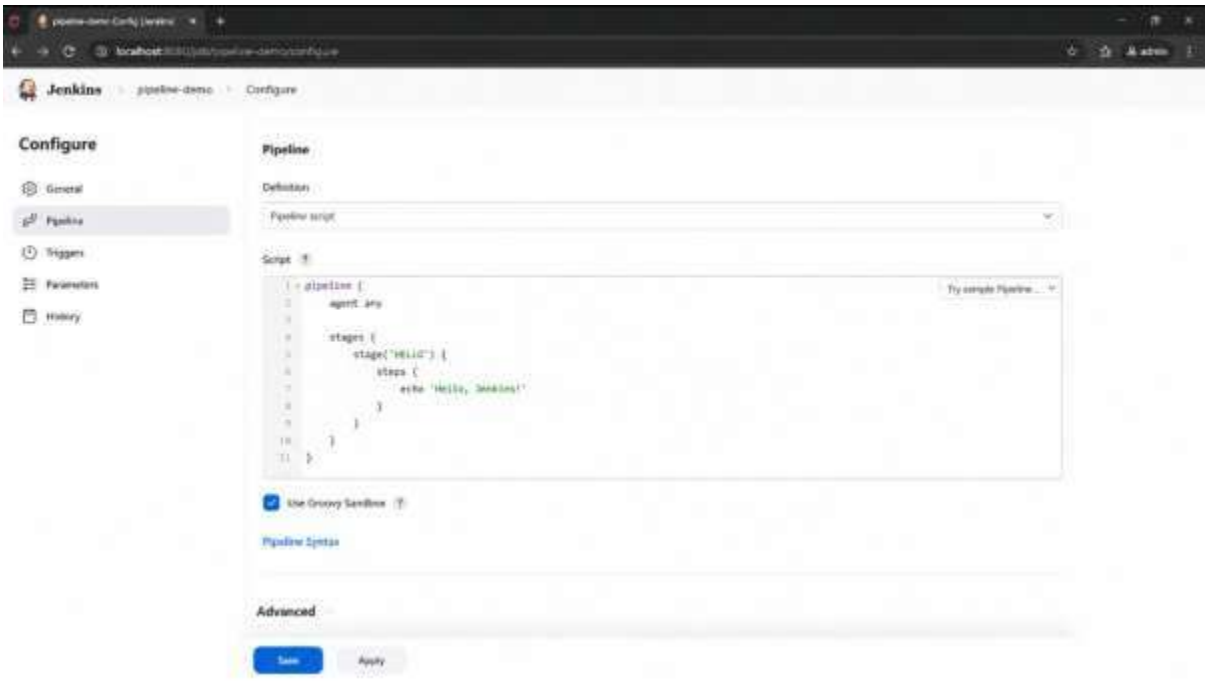


Figure 3.1: Console output of the executed Declarative Pipeline job

3.6.3 Scripted Pipeline — Creation

The Scripted Pipeline was subsequently configured within Jenkins, and its execution was verified as shown below:



Figure 3.2: Configuration and successful build of the Scripted Pipeline

3.7 Result

Both the Declarative and Scripted Jenkins Pipelines were successfully created and executed. The console output for each pipeline confirmed that the build stages ran as defined, validating the correctness of the pipeline configurations.

3.8 Conclusion

The pipeline execution demonstrated a practical understanding of how Jenkins Pipelines can be employed to automate the build, test, and deployment stages of a software project. Through the implementation of both Declarative and Scripted pipeline examples, the structural differences, respective benefits, and appropriate use cases of each syntax style were examined in detail. The obtained output confirms that Jenkins enables the creation of efficient, repeatable, and reliable CI/CD workflows suitable for real-world software development environments.

4. Overall Conclusion

This laboratory exercise provided comprehensive practical exposure to the fundamental concepts and tools associated with Continuous Integration and Continuous Delivery. In Experiment 4.1, the installation and configuration of the Jenkins automation server was carried out, establishing the foundational environment required for automated software delivery. In Experiment 4.2, this foundation was extended through the design and execution of Jenkins Pipelines, encompassing both Declarative and Scripted syntax styles.

The experiment as a whole reinforced the significance of automation in modern software engineering practice, demonstrating how tools such as Jenkins can be leveraged to streamline the build, test, and deployment stages of the software development lifecycle. The skills acquired through this exercise form an essential foundation for the implementation of scalable and reliable CI/CD pipelines in professional DevOps environments.